ESOFT METRO CAMPUS • FACULTY OF COMPUTING
BIT (Hons) in Software Engineering Degree Programme

Web-Based Hotel Management System

Case Study: Dutch Point Negombo Boutique Resort

Project Interim Viva & Demonstration (55% - 60% Milestone Completion)

I.D.P.N Iddamalgoda
ID: E269559
Lead Developer & System Admin
Architecture & Concurrency

H.M.Y.R Herath
ID: E267614
Backend Software Engineer
ASP.NET Core Web API & LINQ

Y.M.A.Y Bandara
ID: E267609
Frontend Software Engineer
React SPA & Booking UI

A.G.H.H Jinadasa
ID: E267612
QA & Requirements Engineer
Requirements Traceability & Testing

Background & The Operational Problem in Negombo

1. Boutique Resort Context

- **Location:** Dutch Point Negombo is a boutique beachfront resort in Western Province, Sri Lanka.
- **Inventory:** Operates luxury accommodations (Deluxe, Ocean View Suites, Family Villas).
- **Guest Demographic:** Serves both international tourists and domestic weekend travelers.
- **Current Operation:** Historically relied on handwritten paper registers, phone calls, and WhatsApp messages for booking rooms.

2. Critical Flaws Identified

- **Frequent Double Bookings:** Front-desk shifts overlapping led to two guests booking the same suite on the same night.
- **Severe Commission Loss:** Third-party OTAs (Booking.com / Agoda) charge 15% - 20% commission on every reservation.
- **No Real-Time Visibility:** Guests calling reception had to wait 10+ minutes for staff to physically search through paper ledgers.
- **Lost Guest Records:** Handwritten cards often misplaced, causing disputes over headcounts, special requests, and room rates.

3. Currency & Localization

- **Foreign Currency Disconnect:** Commercial software quotes rates in USD ($), forcing confusing bank conversion rates on Sri Lankan guests.
- **Lack of Direct Settlement:** Hotel management demanded all rates and invoices be localized strictly in Sri Lankan Rupees (LKR).
- **Headcount Safety:** Guests booking 2-person suites were arriving with 4–5 guests without capacity validation.
- **The Solution Needed:** A dedicated, web-based hotel management system built specifically for Dutch Point Negombo's workflow.

PROJECT GOALS & SCOPE

Project Aims & Business Objectives

[PRESENTATION TIER] **Objective 1: Direct Commission-Free Bookings**

Develop a responsive Single Page Application (React) allowing travelers to browse suites, check real-time availability, and reserve directly with zero OTA commission deductions.

[BACKEND CONCURRENCY] **Objective 2: 100% Prevention of Double Bookings**

Design an atomic transactional check inside ASP.NET Core 8 Web API that mathematically verifies date intervals before saving, returning HTTP 409 Conflict upon collision.

[SECURITY & IDENTITY] **Objective 3: Secure Role-Based Access Control (RBAC)**

Implement isolated user access claims (Admin, Receptionist, Customer) to ensure public visitors cannot escalate privileges while staff manage operations safely.

[BUSINESS LOCALIZATION] **Objective 4: Currency Localization & Capacity Safety**

Standardize all room prices in Sri Lankan Rupees (LKR 28,000 to LKR 140,000) and enforce guest headcount validation (Capacity >= Guests) on all reservations.

[HOTEL OPERATIONS] **Objective 5: Operational Front-Desk Occupancy Tracker**

Equip front-desk staff with an interactive live Occupancy Tracker providing single-day occupancy inspection and full resort booking roster synchronization.

Literature Review & Industrial Systems Comparison

Cloudbeds PMS

Commercial Cloud PMS

Target Market:

Mid-to-large international chains

Pricing Model:

High monthly subscription ($150+ / mo)

Commission:

Zero direct, but adds channel manager fees

Currency:

Defaults to USD / EUR (Conversion friction)

Downside:

Cost-prohibitive for small boutique hotels

Oracle Opera Cloud

Enterprise Resort PMS

Target Market:

5-star multi-national resorts

Pricing Model:

Enterprise licensing ($1,000s + setup)

Hardware:

Requires dedicated enterprise infrastructure

Complexity:

Steep learning curve; weeks of staff training

Downside:

Massively over-engineered for 10-25 rooms

Booking.com Extranet

OTA Channel Platform

Target Market:

Global booking visibility

Pricing Model:

No monthly fee, but heavy 15-20% cut

Overbooking:

Sync delays cause frequent double bookings

Control:

Resort has zero ownership of guest data

Downside:

Eats significant portion of hotel profits

Dutch Point PMS (Proposed)

Custom In-House Solution

Target Market:

Tailored for Dutch Point Negombo

Pricing Model:

One-time internal build; ZERO subscriptions

Commission:

100% Commission-Free direct bookings

Currency:

Natively in Sri Lankan Rupees (LKR)

Advantage:

Direct database control, lightweight & fast

Requirements Engineering (Lecturer's 5-Stage SFE Process)

STAGE 1: 1. Stakeholder Identification

Identified 4 primary user groups: Resort General Manager, Front Desk Receptionists, Guests/Tourists, and System Administrator. Formulated Table 5 Stakeholder Profiles.

STAGE 2: 2. Requirements Gathering

Collected operational needs using 4 proven techniques: Semi-Structured Interviews with GM, Historical Logbook Audits, Direct Front-Desk Observation, and 25 Guest Surveys.

STAGE 3: 3. Requirements Analysis

Examined findings to resolve conflicting needs and prioritize essential scope using the MoSCoW Framework (Must, Should, Could, Won't Have) in Table 6.

STAGE 4: 4. Requirements Specification

Documented unambiguous functional rules adhering to IEEE 830 standards, Agile User Stories ('As a [role], I want [feature] so that [benefit]'), and measurable Acceptance Criteria.

STAGE 5: 5. Requirements Validation

Verified specifications before writing code via Formal Supervisor Walkthroughs, Throwaway UI Mockup testing with receptionists, and the Requirements Traceability Matrix (RTM).

Requirements Analysis: MoSCoW Prioritization

MUST HAVE (Mandatory for Interim)

- Secure User Authentication & JWT Claims
- Multi-Variable Room Search (Dates & Capacity)
- Transactional Double-Booking Prevention (409 Conflict)
- Room Pricing Displayed Strictly in Sri Lankan Rupees (LKR)
- Customer Reservation Booking Creation & Persistence

SHOULD HAVE (Highly Desirable)

- Interactive Occupied Rooms Tracker for Front Desk
- Guest Headcount Filter & Overcapacity Rejection
- Real-Time Dynamic Room Status Badges (Available / Occupied)
- Administrative Room Inventory Management (Add/Edit Suites)
- Front-Desk Booking Cancellation & History Roster

COULD HAVE (Nice to Have)

- Housekeeping Cleaning Status Tags (Clean / Dirty)
- Special Guest Request Notes field on Checkout
- Automated SMS Alerts upon Booking Confirmation
- Multi-photo Room Image Gallery Carousel
- Seasonal Promotional Discount Banners

WON'T HAVE (Deferred to Final Submission)

- Live IPG Payment Gateway (PayHere Sandbox held for 100% demo)
- Automated PDF Invoice & Printable Receipt Generation
- Automated Email Server SMTP Dispatching
- Predictive Occupancy Machine Learning Analytics
- Multi-property Hotel Group Management

Three-Tier Enterprise System Architecture

Tier 1: Presentation Tier

Client-Side User Interface

- **Core Framework:** React 18 + Vite SPA Client
- **UI & Styling:** Bootstrap 5 + Custom Boutique Coastal Theme
- **State Management:** React Hooks (useState, useEffect, custom context)
- **Communication:** Asynchronous HTTP REST calls via fetch() API
- **Key Screens:** Guest Search Portal, Booking Modal, Front-Desk Occupancy Tracker, Admin Dashboard

Tier 2: Application Tier

Business Logic & REST API

- **Backend Framework:** ASP.NET Core 8 Web API (C#)
- **Data Access ORM:** Entity Framework Core (EF Core 8)
- **Core Controllers:** AuthController, RoomsController, BookingsController, DashboardController
- **Key Algorithm:** Double-Booking Transactional Overlap Prevention
- **Security Controls:** CORS policy, model validation, parameterized queries, RBAC claim validation

Tier 3: Data Tier

Relational Database Management

- **Database Engine:** MySQL 8.0 Relational Server (Port 3306)
- **Database Schema:** hotel_db (Users, Rooms, Bookings, Payments)
- **Referential Integrity:** Strict Foreign Key constraints (CASCADE / RESTRICT)
- **Currency Precision:** decimal(10,2) for exact Sri Lankan Rupee calculations
- **Optimization:** Indexed columns on CheckInDate, CheckOutDate, RoomId for sub-50ms query latency

Database Design & Data Dictionary (hotel_db)

Table: Users

UserId (PK)

INT AUTO_INCREMENT - Unique identifier

FullName

VARCHAR(100) - Customer or staff name

Email (UNIQUE)

VARCHAR(100) - Login identity

PasswordHash

VARCHAR(255) - Secure credentials

Role

VARCHAR(20) - Admin / Customer

Phone

VARCHAR(20) - Contact phone number

Table: Rooms

RoomId (PK)

INT AUTO_INCREMENT - Unique room ID

RoomNumber (UQ)

VARCHAR(10) - e.g., Room 101, 201

RoomType

VARCHAR(50) - Deluxe, Ocean Suite

PricePerNight

DECIMAL(10,2) - Price in LKR

Capacity

INT - Max headcount (2, 4)

Status

VARCHAR(20) - Available / Maintenance

Table: Bookings

BookingId (PK)

INT AUTO_INCREMENT - Reservation number

CustomerId (FK)

INT - References Users(UserId)

RoomId (FK)

INT - References Rooms(RoomId)

CheckInDate

DATE - Stay arrival date

CheckOutDate

DATE - Stay departure date

NumberOfGuests

INT - Enforced <= Capacity

TotalAmount

DECIMAL(10,2) - Total Stay in LKR

BookingStatus

VARCHAR(30) - Confirmed / Cancelled

Table: Payments

PaymentId (PK)

INT AUTO_INCREMENT - Transaction ID

BookingId (FK)

INT - References Bookings

Amount

DECIMAL(10,2) - Settlement amount LKR

PaymentMethod

VARCHAR(30) - Cash / Card / Transfer

PaymentStatus

VARCHAR(20) - Completed / Pending

PaidAt

DATETIME - Timestamp of payment

Core Technical Highlight: Concurrency & Overlap Algorithm

The Double-Booking Problem

• **The Challenge:** If two guests book Room 101 simultaneously, a race condition could accept both, humiliating hotel staff when guests arrive.

• **Mathematical Rule:** Two date intervals [A_in, A_out] and [B_in, B_out] overlap IF AND ONLY IF:

(ExistingCheckIn < RequestedCheckOut) AND (ExistingCheckOut > RequestedCheckIn).

• **Server-Side Enforcement:** Validation is never trusted to the frontend. The check executes inside an ACID database transaction in ASP.NET Core prior to saving.

• **Immediate Rejection:** If an overlap exists, the backend instantly rejects the request with HTTP 409 Conflict, returning the existing booking ID and dates.

• **Clean Cancellation:** Cancelled bookings are ignored by the filter, immediately freeing up dates for other travelers.

C# / EF Core LINQ Implementation

Security Hardening & Scope Baseline Adjustments

Change 1: Broken Access Control Fixed

- **Issue Identified:**

Initial prototype had a dropdown on the signup modal where any public user could select 'Admin'.

- **Security Vulnerability:**

OWASP Top 10 - Broken Access Control & Privilege Escalation.

- **Resolution:**

Completely removed client-side role dropdown. Public registrations are strictly hardcoded to 'Customer' on the backend.

- **Admin Isolation:**

Dedicated Admin credentials (username: Pasindu) are managed directly in the MySQL database.

Change 2: Currency Localization to LKR

- **Issue Identified:**

Early proposal drafts used US Dollars ($), leading to exchange rate confusion.

- **Stakeholder Need:**

Dutch Point Negombo management required all billing in Sri Lankan Rupees for accurate daily reconciliation.

- **Resolution:**

Migrated all pricing models, API calculations, and UI displays to Sri Lankan Rupees (LKR 28,000 to LKR 140,000 / night).

- **Impact:**

Clean local guest transactions and direct matching with commercial bank deposits.

Change 3: Guest Headcount & Safety

- **Issue Identified:**

Guests were booking 2-person suites for 5 adults without warning, violating resort safety regulations.

- **Resolution:**

Added dynamic Guest Count selector to search bar. The backend enforces Capacity >= Guests upon booking.

- **UI Feedback:**

Rooms with insufficient capacity are automatically filtered out of search results.

- **Verification:**

Verified via Test Case TC-09 (Overcapacity Headcount Rejection).

INTERIM SYSTEM DEMO

Practical Demonstration Highlights (What We Show Live)

Step 1: Public Registration & RBAC

Demonstrate registering 'Kasun Silva' as a Customer. Prove via MySQL Workbench that Kasun is inserted with Role='Customer', and admin menu items remain hidden.

Step 2: Availability & Capacity Search

Select stay dates (Oct 10-12, 2026) and 2 Guests. Demonstrate dynamic green 'AVAILABLE' badge and LKR rate calculation (2 nights * LKR 28,000 = LKR 56,000).

Step 3: Live Booking Persistence

Complete reservation for Deluxe Suite 101. Show instant generation of Reservation ID (RES-1001) and query MySQL Bookings table to prove row insertion.

Step 4: Live Double-Booking Prevention

Attempt to book the identical Suite 101 for overlapping dates (Oct 11-14). Show immediate UI rejection and inspect F12 DevTools returning HTTP 409 Conflict.

Step 5: Front-Desk Occupancy Tracker

Open Occupancy Tracker. Demonstrate single-date occupancy query (showing Suite 101 Occupied) and toggle to 'All Active Bookings' showing the full resort roster.

Step 6: Admin Dashboard & Revenue KPIs

Log in as Admin ('Pasindu'). Show dynamic appearance of Admin Dashboard, live revenue tracking in LKR, and active reservation management tools.

System Verification & Traceability Matrix (RTM)

[TC-01] Valid Room Booking | Expected: Booking saved, total LKR calculated, RES ID issued **[PASS]**

[TC-02] Past Date Rejection | Expected: Frontend & API block submission with date error **[PASS]**

[TC-03] Reversed Dates Rejection | Expected: Validation error: Check-out must follow check-in **[PASS]**

[TC-04] Double-Booking Overlap | Expected: Rejected with HTTP 409 Conflict, no DB write **[PASS]**

[TC-05] Customer Role Isolation | Expected: Assigned 'Customer' role, admin menus hidden **[PASS]**

[TC-06] Admin Dashboard Access | Expected: Admin navbar unlocked, revenue stats displayed **[PASS]**

[TC-07] Reservation Cancellation | Expected: Status updated to 'Cancelled', room freed up **[PASS]**

[TC-08] Occupancy Tracker Sync | Expected: Displays occupied suites, guest name, contact info **[PASS]**

[TC-09] Overcapacity Rejection | Expected: Search filters room out; API rejects with HTTP 400 **[PASS]**

Project Management & Milestone Completion (55%–60%)

Completed Modules (55% - 60%)

- ✓ **Stakeholder Identification & Elicitation: 100% Complete**
- ✓ **Literature Review & Technology Benchmarking: 100% Complete**
- ✓ **Database Relational Schema (hotel_db): 100% Complete**
- ✓ **Three-Tier ASP.NET Core Web API: 100% Complete**
- ✓ **React Single Page Application Client: 100% Complete**
- ✓ **Double-Booking Concurrency Algorithm: 100% Complete**
- ✓ **Currency Localization to LKR: 100% Complete**
- ✓ **Interactive Front-Desk Occupancy Tracker: 100% Complete**
- ✓ **Admin KPI Revenue Dashboard: 80% Complete**

Milestone Tracking & Schedule Status

- **Phase 1: Project Proposal & Scope Approval**
  Timeline: Weeks 1 - 3 | Status: **Completed on Schedule**
- **Phase 2: Requirements Engineering (SFE 5 Stages)**
  Timeline: Weeks 4 - 6 | Status: **Completed on Schedule**
- **Phase 3: Database & Core Backend Architecture**
  Timeline: Weeks 7 - 9 | Status: **Completed on Schedule**
- **Phase 4: Frontend Development & Concurrency**
  Timeline: Weeks 10 - 12 | Status: **Completed on Schedule**
- **Phase 5: Interim Report & System Demo (55-60%)**
  Timeline: Current Viva | Status: **Delivered Today**
- **Phase 6: Payment Gateway, PDF & SMS Alerts**
  Timeline: Weeks 13 - 16 | Status: **Final Semester Target**
- **Phase 7: Final Dissertation & Live Defense**
  Timeline: Weeks 17 - 18 | Status: **Final Submission Target**

FUTURE IMPLEMENTATION

Final Semester Roadmap (Remaining 40% to 100%)

Module 1: Payment Gateway (IPG)

- **Objective:**

Integrate PayHere Sandbox payment gateway for online credit/debit card processing.

- **Security:**

Tokenized card transactions, webhook signature verification, and HTTPS encryption.

- **Outcome:**

Eliminates cash-only restrictions and guarantees prepaid revenue for the hotel.

Module 2: Automated PDF Invoicing

- **Objective:**

Implement automated PDF invoice and receipt generation upon booking confirmation.

- **Features:**

Official Dutch Point Negombo letterhead, tax breakdown, guest details, and unique QR code.

- **Outcome:**

Allows guests and corporate travelers to instantly download travel expense receipts.

Module 3: SMS & Email Notifications

- **Objective:**

Connect Twilio / Dialog SMS Gateway and SMTP email dispatcher.

- **Triggers:**

Immediate SMS dispatch upon reservation, 24-hour pre-arrival reminders, and cancellation alerts.

- **Outcome:**

Reduces guest no-shows and eliminates telephone confirmation delays.

Module 4: Business Intelligence Analytics

- **Objective:**

Build advanced operational analytics for resort management.

- **Metrics:**

Monthly RevPAR (Revenue Per Available Room), Average Daily Rate (ADR), and occupancy heatmaps.

- **Outcome:**

Empowers hotel management with data-driven peak-season pricing strategies.

DUTCH POINT NEGOMBO • HOTEL MANAGEMENT SYSTEM

Summary of Interim Outcomes

✓ Successfully built an enterprise Three-Tier solution (React SPA + ASP.NET Core 8 Web API + MySQL).

✓ Mathematically eradicated double bookings via transactional server-side LINQ overlap checks (HTTP 409).

✓ Fully localized pricing into Sri Lankan Rupees (LKR) and enforced guest headcount capacity limits.

✓ Delivered an intuitive front-desk Occupancy Tracker with single-day filter and full booking roster views.

✓ Project is strictly on schedule at 55% - 60% completion, prepared for Final Semester implementation.

Thank you! We welcome questions from the examination panel.